

```
!pip install gradio
```

[Show hidden output](#)

```
!pip install gtts
```

[Show hidden output](#)

```
!pip install typer==0.9.0 click==8.1.3 typing_extensions --force-reinstall
```

[Show hidden output](#)

```
import gradio as gr
import random
import matplotlib.pyplot as plt
from gtts import gTTS
import tempfile

def collect_data():
    return {
        "traffic": random.randint(1, 10),
        "weather": random.randint(1, 10),
        "accidents": random.randint(10, 100),
        "road": random.randint(1, 10)
    }

def calculate_risk(data):
    return round(
        0.4 * data["accidents"]/100 +
        0.3 * data["traffic"]/10 +
        0.2 * data["weather"]/10 +
        0.1 * data["road"]/10,
        2
    )

def get_route_name(source, destination):
    roads = [
        f"{source} → NH48 → {destination}",
        f"{source} → Ring Road → {destination}",
        f"{source} → Expressway → {destination}",
        f"{source} → City Bypass → {destination}",
        f"{source} → State Highway → {destination}"
    ]
    return random.choice(roads)

def get_weather(location):
    conditions = ["Sunny", "Cloudy", "Rainy", "Stormy"]
    return {
        "location": location,
        "condition": random.choice(conditions),
        "temperature": random.randint(18, 38),
        "humidity": random.randint(40, 90),
        "wind_speed": random.randint(5, 40)
    }

def generate_graph(routes):
    names = [r["name"] for r in routes]
    risks = [r["risk"] for r in routes]

    plt.figure(figsize=(10,6))
    plt.plot(names, risks, marker='o')
    plt.xlabel("Routes")
    plt.ylabel("Risk Score")
    plt.title("Route Risk Comparison")
    plt.grid()

    return plt

def generate_llm_response(routes, preference, source_weather, destination_weather):

    safest = min(routes, key=lambda x: x["risk"])
    fastest = max(routes, key=lambda x: x["data"]["traffic"])

    response = ""

    for i, r in enumerate(routes):
        response += f"{i+1} Route: {r['name']} Risk: {r['risk']}\n"
```

```

response += f" \n\nRoute {chr(ord+1)}:\n"
response += f" Path: {r['route_path']}\n" # ★ NEW LINE
response += f" Risk Score: {r['risk']}\n"
response += f" Traffic: {r['data']['traffic']}\n"
response += f" Weather: {r['data']['weather']}\n"
response += f" Accidents: {r['data']['accidents']}\n"

response += f"\n Condition: {r['weather']['condition']}\n"
response += f" Temperature: {r['weather']['temperature']}°C\n"
response += f" Humidity: {r['weather']['humidity']}%\n"
response += f" Wind Speed: {r['weather']['wind_speed']} km/h\n"

if r['weather']['condition'] == "Stormy":
    response += f" ⚠ DANGER ALERT: Storm ahead in {r['name']}\n"
elif r['risk'] > 0.6:
    response += f" ⚠ HIGH RISK: Accident-prone route detected\n"

response += "\n\n🧠 Intelligent Analysis:\n"
if preference == "Safe":
    response += f"I recommend {safest['name']} because it is safest.\n"
elif preference == "Fast":
    response += f"I recommend {fastest['name']} because it is fastest.\n"
else:
    response += f"I recommend {safest['name']} because it provides a balanced route.\n"

response += "\n📍 Source Weather:\n"
response += f"Location: {source_weather['location']}\n"
response += f"Condition: {source_weather['condition']}\n"
response += f"Temperature: {source_weather['temperature']}°C\n"
response += f"Humidity: {source_weather['humidity']}%\n"
response += f"Wind Speed: {source_weather['wind_speed']} km/h\n"

response += "\n📍 Destination Weather:\n"
response += f"Location: {destination_weather['location']}\n"
response += f"Condition: {destination_weather['condition']}\n"
response += f"Temperature: {destination_weather['temperature']}°C\n"
response += f"Humidity: {destination_weather['humidity']}%\n"
response += f"Wind Speed: {destination_weather['wind_speed']} km/h\n"

response += "\n⚠ Safety Insights:\n"
response += "- High traffic increases collision probability.\n"
response += "- Bad weather increases braking distance.\n"
response += "- Wind affects vehicle stability.\n"

return response

def text_to_speech(text):
    tts = gTTS(text)
    temp_file = tempfile.NamedTemporaryFile(delete=False, suffix=".mp3")
    tts.save(temp_file.name)
    return temp_file.name

def generate_map(source, destination):
    return f"""
<div style="width:100%; font-family:sans-serif; background:#111; color:white;">

    <!-- 🗺 HEADER -->
    <div style="padding:10px;">
        <h3>🗺 Explore Nearby</h3>
    </div>

    <!-- 🍽 FOOD & DRINK -->
    <div style="padding:10px;">
        <b>Food & Drink</b><br><br>
        <button onclick="searchPlace('restaurants')">🍽 Restaurants</button>
        <button onclick="searchPlace('takeout')">🍷 Takeout</button>
        <button onclick="searchPlace('bars')">🍸 Bars</button>
        <button onclick="searchPlace('delivery food')">🛵 Delivery</button>
        <button onclick="searchPlace('coffee')">☕ Coffee</button>
    </div>

    <!-- 🎉 THINGS TO DO -->
    <div style="padding:10px;">
        <b>Things to do</b><br><br>
        <button onclick="searchPlace('parks')">🌳 Parks</button>
        <button onclick="searchPlace('gym')">🏋️ Gyms</button>
        <button onclick="searchPlace('art gallery')">🎨 Art</button>
        <button onclick="searchPlace('tourist attractions')">📍 Attractions</button>
        <button onclick="searchPlace('night clubs')">🎵 Nightlife</button>
        <button onclick="searchPlace('live music')">🎸 Live Music</button>
    </div>
    """

```

```

        <button onclick="searchPlace('cinema')">🎬 Films</button>
        <button onclick="searchPlace('museum')">🏛️ Museums</button>
        <button onclick="searchPlace('library')">📖 Libraries</button>
    </div>

    <!-- 🛒 SHOPPING -->
    <div style="padding:10px;">
        <b>Shopping</b><br><br>
        <button onclick="searchPlace('grocery store')">🛒 Groceries</button>
        <button onclick="searchPlace('shopping mall')">🏬 Shopping Centers</button>
        <button onclick="searchPlace('beauty store')">💄 Beauty Supplies</button>
        <button onclick="searchPlace('electronics store')">💻 Electronics</button>
        <button onclick="searchPlace('car dealer')">🚗 Car Dealers</button>
        <button onclick="searchPlace('sports store')">🏈 Sporting Goods</button>
        <button onclick="searchPlace('home goods store')">🏠 Home & Garden</button>
        <button onclick="searchPlace('clothing store')">👕 Clothing</button>
    </div>

    <!-- 🏠 SERVICES -->
    <div style="padding:10px;">
        <b>Services</b><br><br>
        <button onclick="searchPlace('food shelter')">🍲 Food Shelters</button>
        <button onclick="searchPlace('dry cleaning')">🧺 Dry Cleaning</button>
        <button onclick="searchPlace('night shelter')">🌙 Night Shelters</button>
        <button onclick="searchPlace('charging station')">⚡ Charging Station</button>
        <button onclick="searchPlace('hotel')">🏨 Hotels</button>
        <button onclick="searchPlace('petrol pump')">🛢️ Petrol</button>
        <button onclick="searchPlace('atm')">🏧 ATM</button>
        <button onclick="searchPlace('hospital')">🏥 Hospitals</button>
        <button onclick="searchPlace('beauty salon')">💇 Beauty Salon</button>
        <button onclick="searchPlace('post office')">📬 Post</button>
        <button onclick="searchPlace('car rental')">🚗 Car Hire</button>
        <button onclick="searchPlace('parking')">🅿️ Parking</button>
        <button onclick="searchPlace('car repair')">🔧 Car Repair</button>
        <button onclick="searchPlace('pharmacy')">💊 Chemists</button>
    </div>

    <!-- 🗺️ MAP TYPE -->
    <div style="padding:10px;">
        <b>🗺️ Map Type</b><br><br>
        <button onclick="changeMap('roadmap')">Default</button>
        <button onclick="changeMap('satellite')">Satellite</button>
        <button onclick="changeMap('terrain')">Terrain</button>
    </div>

    <!-- 📍 MAP -->
    <iframe id="mapframe"
        width="100%"
        height="500"
        style="border:0;"
        src="https://www.google.com/maps?q={source}+to+{destination}&output=embed">
    </iframe>

</div>

<script>
    function searchPlace(place) {{
        document.getElementById("mapframe").src =
            "https://www.google.com/maps?q=" + place + "+near+{source}&output=embed";
    }}

    function changeMap(type) {{
        let url = "https://www.google.com/maps?q={source}+to+{destination}&output=embed";

        if(type === "satellite") {{
            url += "&t=k";
        }} else if(type === "terrain") {{
            url += "&t=p";
        }}

        document.getElementById("mapframe").src = url;
    }}
</script>
"""
def predict(source, destination, preference):

    routes = []

    for i in range(3):
        data = collect_data()

```

```

risk = calculate_risk(data)

routes.append({
    "name": f"Route {chr(65+i)}",
    "route_path": get_route_name(source, destination), # 🌟 NEW
    "data": data,
    "risk": risk,
    "weather": get_weather(f"Route {chr(65+i)}")
})

source_weather = get_weather(source)
destination_weather = get_weather(destination)

text_output = generate_llm_response(routes, preference, source_weather, destination_weather)
map_output = generate_map(source, destination)
graph_output = generate_graph(routes)

audio_output = text_to_speech(text_output)

return text_output, map_output, graph_output, audio_output

app = gr.Interface(
    fn=predict,
    inputs=[
        gr.Textbox(label="Enter Source"),
        gr.Textbox(label="Enter Destination"),
        gr.Dropdown(["Safe", "Fast", "Balanced"], label="Preference")
    ],
    outputs=[
        gr.Textbox(label="AI Navigation Output", lines=25),
        gr.HTML(label="Map View"),
        gr.Plot(label="Route Risk Graph"),
        gr.Audio(label="🔊 Auto Speech Output", autoplay=True)
    ],
    title="🚦 Master GPS",
    description="Navigation That Protects Lives."
)

app.launch()

```

It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share=True`.

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

Could not create share link. Please check your internet connection or our status page: <https://status.gradio.app>.

2026/04/01 15:36:28 [W] [service.go:132] login to server failed: tls: failed to verify certificate: x509: certificate has expired or is not yet valid: current time 2026-04-01T15:36:28Z is after 2026-03-31T23:59:59Z

Running on <https://localhost:7860/>

🚦 Master GPS

Navigation That Protects Lives.

Enter Source

Jaipur

Enter Destination

Bhopal

Preference

Safe

Clear

Submit

AI Navigation Output

Route A:
Path: Jaipur → Expressway → Bhopal
Risk Score: 0.66
Traffic: 5
Weather: 6
Accidents: 95

Condition: Rainy
Temperature: 36°C
Humidity: 44%
Wind Speed: 29 km/h
⚠️ HIGH RISK: Accident-prone route detected

